HW #4.3: Reproducible Data Analysis

Pragya Sinha

2026-04-15

Busiest Airports Analysis

My Rationale

To analyze global air travel trends, I examined passenger traffic data for six major international airports from 2020 to 2025. I created both a table and a graph to compare both changes over time and differences between airports.

Table 1: Passenger traffic for six busy airports from 2020 to 2025.

Airport	2020	2021	2022	2023	2024	2025
Beijing Daxing	25,051,012	49,441,029	53,618,949	NA	NA	NA
Dallas Fort Worth	62,465,756	87,817,864	85,660,127	39,364,990	73,362,946	81,755,538
Dubai International	29,110,609	92,331,506	95,200,000	25,836,771	66,069,981	86,994,365
Frankfurt	24,812,849	61,564,957	63,189,666	18,768,601	48,918,482	59,355,389
Hartsfield-Jackson Atlanta	75,704,760	108,067,766	106,302,208	42,918,685	93,699,630	104,653,451
Tokyo Haneda	25,876,429	85,900,167	91,430,000	31,055,210	50,290,705	78,719,302

Passenger traffic trends for six busy airports from 2020 to 2025.

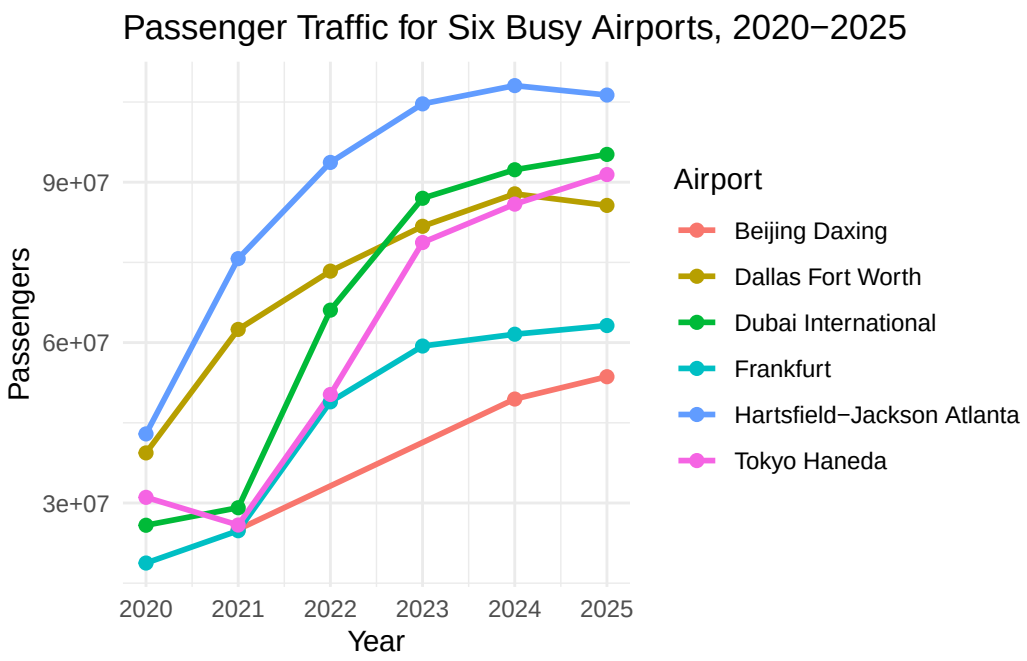


Figure 1: Passenger traffic trends for six busy airports from 2020 to 2025.

My Findings

Table 1 provides exact passenger counts for each airport and year, allowing for precise comparisons. Figure 1 complements this by clearly showing trends and patterns over time.

One major pattern that I saw was that all airports show an increase in passenger traffic after 2020. This is likely due to a recovery of global air travel following a steep decline in that year. Hartsfield-Jackson Atlanta Airport consistently has the highest passenger numbers across all years. The graph I created shows a strong increase early on, followed by a slower rate of growth, suggesting a strong and early recovery.

Dubai International Airport and Tokyo Haneda Airport also show strong increases, particularly after 2021, indicating a strong recovery in those regions. Dallas Fort Worth International Airport shows steady growth across all years but appears to fall off slightly by 2025.

Frankfurt Airport has a lower passenger count compared to the others and grows more gradually. Beijing Daxing International Airport starts with lower passenger numbers and has some missing early data, but still shows steady growth in later years.

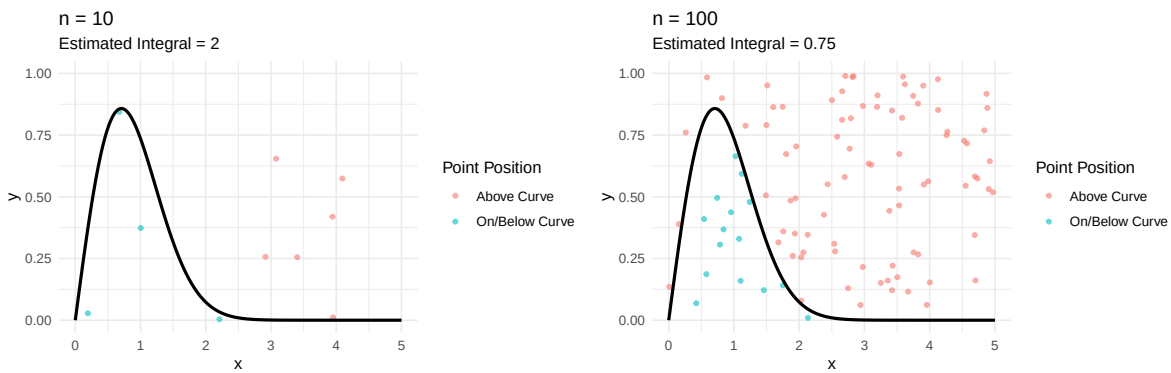
Overall, using both the table and graph provided me with a clearer understanding of the data. The table allows for precise comparisons, while the graph highlights overall trends such as

recovery and growth. Together, they show that although all airports experienced increases after 2020, the rate of growth and total scale of recovery varied across locations.

Monte Carlo Numerical Integration

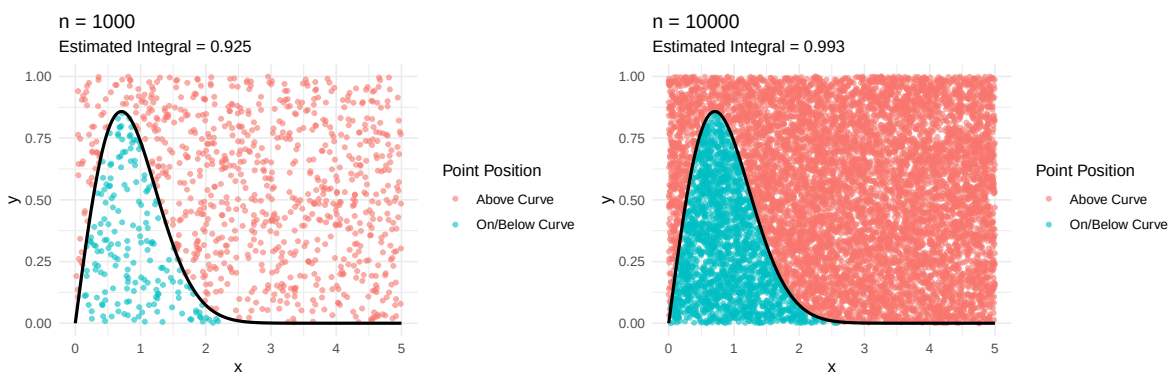
My Rationale

Monte Carlo numerical integration is a method used to estimate the area under a curve using random sampling. Instead of calculating the integral directly, random points are generated within a defined region, and the proportion of points that fall under the curve is used to estimate the total area.



(a) Monte Carlo estimate with n = 10.

(a) Monte Carlo estimate with n = 100.



(a) Monte Carlo estimate with n = 1000.

(a) Monte Carlo estimate with n = 10000.

My Findings

The small multiple visualizations show how the accuracy of the estimate improves as the number of sampled points increases. With smaller sample sizes, such as 10 or 100, the estimates vary significantly and are less reliable due to randomness.

As the number of points increases to 1,000 and 10,000, the estimate becomes much more stable and consistent. In the highest-resolution plot, the estimate is close to 1, suggesting that the true value of the integral is exactly 1. The estimates in the plots approach 1 as the sample size increases, which suggests convergence to that exact value.

This result makes sense because the function being integrated is a probability density function, and these functions are defined so that the total area under the curve equals 1. The increasing density of points in the higher-resolution plots provides strong visual evidence that the estimate is converging to the true value.

Planning and Prompting GenAI Tools

Plan

The goal of this section is to tidy the calcium dataset and create a visualization that allows me to compare changes in ulnar calcium levels over time between the null treatment group and the diet/exercise group.

Needs

- calcium.csv dataset
- Variables: Initial, Year 1, Year 2, Year 3 for both groups
- R packages: dplyr, tidyr, ggplot2
- Understanding the tidy data structure
- Visualization method

Steps

1. Load the calcium.csv dataset into R
2. Inspect the dataset to understand its structure and confirm that the data is not tidy
3. Separate the columns into two groups: null treatment and diet/exercise treatment
4. Rename columns so that time points (Initial, Year 1, Year 2, Year 3) are clearly labeled
5. Add a variable that identifies the treatment group (Null or Diet/Exercise)
6. Combine both groups into one dataset
7. Reshape the dataset from wide format to long format using pivot_longer so that each row represents one observation
8. Create a time variable (Initial, Year 1, Year 2, Year 3)
9. Create a line plot using ggplot2:
    - x-axis: time
    - y-axis: calcium measurement
    - color: treatment group
    - group: individual participants (optional)
10. Add labels, title, and a clean theme
11. Interpret the visualization by comparing trends between the two groups over time

GenAI Output

I'm checking the file structure first so the code matches your actual column names and shape.

Thought for 17s

Here's R code that tidies the file and makes the plot you want.

Your CSV has 8 columns:

- `y0`, `y1`, `y2`, `y3` = Null group
- `y0.1`, `y1.1`, `y2.1`, `y3.1` = Diet/Exercise group

It also looks like one row is missing all four Null values, which is why the total ends up being 31 women after tidying.

```
library(dplyr) library(tidyr) library(ggplot2)  # Read the data calcium <- read.csv("calcium
```

How this works:

First, `mutate(row_id = row_number())` creates an ID for each row. That gives each woman a row identifier before reshaping.

Then the code splits the dataset into two parts:

- the first 4 columns for the Null group
- the next 4 columns for the Diet/Exercise group

For each group, `pivot_longer()` changes the data from wide to long format. So instead of one row per woman with 4 separate time columns, you get one row per woman per time point.

Then `mutate(treatment = ...)` adds the treatment group label, and `recode()` converts variable names like `y0` and `y1` into readable time labels:

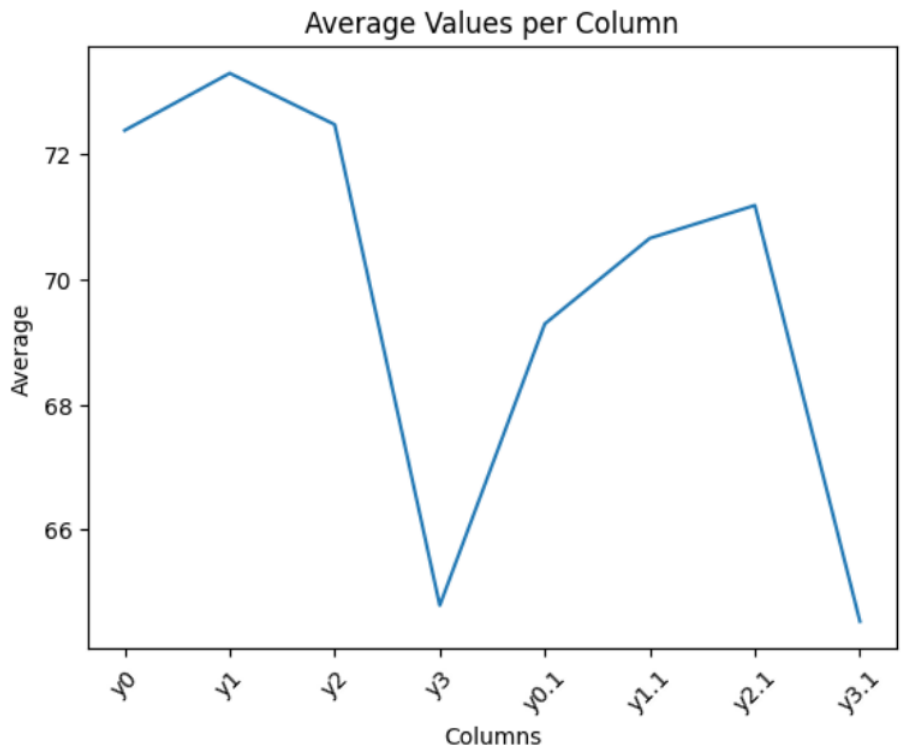
- `Initial`
- `Year 1`
- `Year 2`
- `Year 3`

After that, `bind_rows()` stacks the two long datasets together into one tidy dataset.

Finally, the plot uses a summarized dataset with the mean calcium level for each treatment group at each time point, and `ggplot2` draws a line plot comparing the groups over time.

If you want, I can also give you a version that plots each individual woman's trajectory faintly in the background with the group mean on top.

Generic Prompt Output



The generic prompt asked ChatGPT to "Create a data visualization for these data." The response created a visualization showing the average value of each column in the dataset. Each point represented the mean of one column, allowing for comparison of which variables had higher or lower values overall. The output focused on comparing averages across columns rather than analyzing the structure of the dataset or comparing treatment groups over time. It also suggested alternative visualizations such as boxplots, time series plots, and correlation heatmaps.

Comparison

The plan-informed response and the generic prompt response differ significantly in quality and usefulness. The plan-informed response is much more specific to the calcium dataset. It correctly identifies the structure of the data, separates the two treatment groups, and reshapes the dataset into a tidy format. It also produces a meaningful line plot that allows for comparison of calcium levels over time between the two groups. In contrast, the generic prompt response is much more limited. It focuses only on calculating averages for each column and does not recognize the structure of the dataset or the presence of two treatment groups. As a result, the visualization does not help answer the main question about how calcium levels change over

time between groups. This comparison shows that providing a clear and detailed plan leads to more accurate, useful, and meaningful outputs from generative AI tools. A generic prompt, on the other hand, produces a less relevant and less informative result.

Reflection

Throughout this course, I have learned how to better organize, analyze, and communicate data in a clear and meaningful way. At the beginning, I focused mostly on getting code to run, but now I spend more time planning my approach and thinking about what I want to learn from the data before writing any code. One specific example of this is the planning process we used before creating visualizations. In earlier assignments, I would jump straight into making a plot, but now I understand the importance of defining goals, identifying what I need, and outlining steps first. This helped me create more effective visualizations, such as in the busiest airports analysis where I used both a table and a graph to better understand trends over time. I also improved my ability to interpret data rather than just display it. For example, in the Monte Carlo simulation, I learned how increasing the number of random points leads to more accurate estimates. This helped me understand not just how to create the plot, but what the results actually mean and why they make sense mathematically. Another important skill I developed is working with messy or non-tidy data. In this assignment, I had to think carefully about how to restructure the calcium dataset so that it could be analyzed properly. This reinforced the idea that data preparation is a critical step before visualization. Overall, I have become more confident in both coding and explaining my results. I now focus more on clarity, organization, and interpretation, rather than just producing output. These skills will be useful not only in this course but in any future work involving data analysis.

Appendix A: GenAI Usage

Tool Information:

- **Tool used:** ChatGPT
- **Model:** GPT-5.3
- **Date of usage:** April 15, 2026

Usage 1: Planning and Prompting Section

Prompt used:

I have uploaded a dataset called calcium.csv.

These data come from a longitudinal study of 31 women measured over four years. The dataset is not tidy. The first four columns represent calcium measurements (Initial, Year 1, Year 2, Year 3) for the null treatment group, and the next four columns represent the same measurements for the diet/exercise treatment group.

I want to:
1. Tidy the dataset into a long format
2. Add a variable indicating treatment group (Null vs Diet/Exercise)
3. Create a time variable (Initial, Year 1, Year 2, Year 3)
4. Create a visualization to compare calcium levels over time between the two groups

Please:
- Write R code using dplyr, tidyr, and ggplot2
- Clearly explain each step
- Create a line plot showing calcium levels over time by treatment group

Response generated by the tool:

I'm checking the file structure first so the code matches your actual column names and shape.

Thought for 17s

Here's R code that tidies the file and makes the plot you want.

Your CSV has 8 columns:

- `y0`, `y1`, `y2`, `y3` = Null group
- `y0.1`, `y1.1`, `y2.1`, `y3.1` = Diet/Exercise group

It also looks like one row is missing all four Null values, which is why the total ends up being 31 women after tidying.

```
library(dplyr) library(tidyr) library(ggplot2)  # Read the data calcium <- read.csv("calcium
```

How this works:

First, `mutate(row_id = row_number())` creates an ID for each row. That gives each woman a row identifier before reshaping.

Then the code splits the dataset into two parts:

- the first 4 columns for the Null group
- the next 4 columns for the Diet/Exercise group

For each group, `pivot_longer()` changes the data from wide to long format. So instead of one row per woman with 4 separate time columns, you get one row per woman per time point.

Then `mutate(treatment = ...)` adds the treatment group label, and `recode()` converts variable names like `y0` and `y1` into readable time labels:

- `Initial`
- `Year 1`
- `Year 2`
- `Year 3`

After that, `bind_rows()` stacks the two long datasets together into one tidy dataset.

Finally, the plot uses a summarized dataset with the mean calcium level for each treatment group at each time point, and `ggplot2` draws a line plot comparing the groups over time.

If you want, I can also give you a version that plots each individual woman's trajectory faintly in the background with the group mean on top.

Appendix B: Code Appendix

```
# ----------------------------
# Busiest Airports Analysis Code
# ----------------------------

library(dplyr)
library(tidyr)
library(knitr)
library(ggplot2)

busiest_airports_table <- busiest_airports_tidy %>%
  select(airport, year, passengers) %>%
  pivot_wider(
    names_from = year,
    values_from = passengers
  ) %>%
  arrange(airport) %>%
  mutate(
    airport = recode(
      airport,
      "Hartsfield-Jackson Atlanta International Airport" = "Hartsfield-Jackson Atlanta",
      "Dallas Fort Worth International Airport" = "Dallas Fort Worth",
      "Dubai International Airport" = "Dubai International",
      "Tokyo Haneda Airport" = "Tokyo Haneda",
      "Frankfurt Airport" = "Frankfurt",
      "Beijing Daxing International Airport" = "Beijing Daxing"
    ),
    `2020` = ifelse(is.na(`2020`), "NA", format(round(`2020`, 0), big.mark = ",", scientific
    `2021` = ifelse(is.na(`2021`), "NA", format(round(`2021`, 0), big.mark = ",", scientific
    `2022` = ifelse(is.na(`2022`), "NA", format(round(`2022`, 0), big.mark = ",", scientific
    `2023` = ifelse(is.na(`2023`), "NA", format(round(`2023`, 0), big.mark = ",", scientific
    `2024` = ifelse(is.na(`2024`), "NA", format(round(`2024`, 0), big.mark = ",", scientific
    `2025` = ifelse(is.na(`2025`), "NA", format(round(`2025`, 0), big.mark = ",", scientific
  )

knitr::kable(
  busiest_airports_table,
  caption = "Table 1: Passenger traffic for six busy airports from 2020 to 2025.",
  col.names = c("Airport", "2020", "2021", "2022", "2023", "2024", "2025"),
  align = c("l", "r", "r", "r", "r", "r", "r")
)
```

```r
busiest_airports_plot <- busiest_airports_tidy %>%
  mutate(
    airport = recode(
      airport,
      "Hartsfield-Jackson Atlanta International Airport" = "Hartsfield-Jackson Atlanta",
      "Dallas Fort Worth International Airport" = "Dallas Fort Worth",
      "Dubai International Airport" = "Dubai International",
      "Tokyo Haneda Airport" = "Tokyo Haneda",
      "Frankfurt Airport" = "Frankfurt",
      "Beijing Daxing International Airport" = "Beijing Daxing"
    )
  )

ggplot(
  busiest_airports_plot,
  aes(x = year, y = passengers, color = airport, group = airport)
) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  scale_x_continuous(breaks = 2020:2025) +
  labs(
    title = "Passenger Traffic for Six Busy Airports, 2020-2025",
    x = "Year",
    y = "Passengers",
    color = "Airport"
  ) +
  theme_minimal()

# -----------------------------
# Monte Carlo Simulation Code
# -----------------------------

monte_carlo_points <- function(n, xmin, xmax, ymin, ymax) {
  data.frame(
    x = runif(n, min = xmin, max = xmax),
    y = runif(n, min = ymin, max = ymax)
  )
}

make_mc_plot <- function(n_points) {
  x_min <- 0
  x_max <- 5
```

```r
  y_min <- 0
  y_max <- 1

  mc_points <- monte_carlo_points(n_points, x_min, x_max, y_min, y_max)

  mc_points_wrangled <- mc_points %>%
    mutate(
      curve_y = dweibull(x, shape = 2, scale = 1),
      position = if_else(y <= curve_y, "On/Below Curve", "Above Curve")
    )

  rectangle_area <- (x_max - x_min) * (y_max - y_min)

  estimate_value <- mc_points_wrangled %>%
    summarise(estimate = mean(position == "On/Below Curve") * rectangle_area) %>%
    pull(estimate)

  curve_data <- data.frame(x = seq(x_min, x_max, length.out = 500)) %>%
    mutate(y = dweibull(x, shape = 2, scale = 1))

  ggplot() +
    geom_point(data = mc_points_wrangled,
               aes(x = x, y = y, color = position),
               alpha = 0.6, size = 1.2) +
    geom_line(data = curve_data,
              aes(x = x, y = y),
              linewidth = 1) +
    labs(
      title = paste("n =", n_points),
      subtitle = paste("Estimated Integral =", round(estimate_value, 4)),
      x = "x",
      y = "y",
      color = "Point Position"
    ) +
    coord_cartesian(xlim = c(x_min, x_max), ylim = c(y_min, y_max)) +
    theme_minimal()
}

plot10 <- make_mc_plot(10)
plot100 <- make_mc_plot(100)
plot1000 <- make_mc_plot(1000)
plot10000 <- make_mc_plot(10000)
```

```
plot10
plot100
plot1000
plot10000
```